

Cloud Computing

Complete Revision Notes

Topics Covered

- Service Level Agreements (SLA) — Concepts, Metrics & Calculations
 - Multi-Level SLA Architecture
 - Parallel Databases vs Traditional Databases
 - Real-Time vs Batch Processing
 - Object Storage for Large-Scale Data
 - MapReduce Optimization (Combiners)
 - OpenStack CPU Overcommit
 - Azure SQL Database Compute Tiers
 - Azure VM Disk Limits & Secure Login
 - Google Cloud Database Services
(Cloud Spanner, Bigtable, BigQuery, Cloud SQL)
-

Weeks 3 & 4 — Cloud Computing Course

Exam-Ready • Concept-First • Example-Driven

Contents

1	Service Level Agreement (SLA)	3
1.1	What is an SLA?	3
1.2	Real-Life Examples	3
1.2.1	Internet Service Provider (ISP)	3
1.2.2	Cloud Provider (AWS / Azure / GCP)	3
1.2.3	College IT System (e.g., Campus Portal)	3
1.3	Key Components of an SLA	4
1.4	Understanding Uptime	4
1.5	SLA Calculation — Worked Example	4
1.6	Multi-Level SLA Architecture	5
1.7	Why SLA Matters	6
1.8	Related Concepts: SLO and SLI	6
1.9	Key SLA Metrics	7
2	Parallel Databases vs Traditional Databases	8
2.1	Traditional (Single-Node) Database	8
2.2	Parallel Database	8
2.3	Analogy	8
2.4	Comparison Table	9
3	Real-Time vs Batch Processing	10
3.1	Batch Processing	10
3.2	Real-Time (Stream) Processing	10
3.3	Decision Rule	10
3.4	Comparison Table	11
4	Object Storage for Large-Scale Data	12
4.1	The Problem with Traditional Storage	12
4.2	Object Storage	12
4.3	Examples	12
5	MapReduce and Combiner Optimisation	13
5.1	MapReduce Overview	13
5.2	The Problem: Disk Full During Reduce Phase	13
5.3	Solution: The Combiner	13
6	OpenStack CPU Overcommit	15
6.1	What is CPU Overcommit?	15
6.2	The Formula	15
6.3	Worked Example	15
6.4	Analogy	15
7	Azure SQL Database Compute Tiers	17
7.1	Overview	17
7.2	General Purpose Tier	17
7.3	Business Critical Tier	17
7.4	Hyperscale Tier	17
7.5	Comparison Table	18
8	Azure Virtual Machines — Disk Limits & Secure Access	19

8.1	VM Disk Limit — What Controls It?	19
8.2	Secure Login to Linux VMs	19
8.2.1	Traditional Approach (Less Secure)	19
8.2.2	Best Practice: Azure AD SSH + Azure Bastion	19
8.3	Azure App Service Deployment Slots	20
8.3.1	What are Deployment Slots?	20
8.3.2	Slot Limits by Plan	20
8.3.3	Traffic Warm-Up (Premium Feature)	20
9	Azure Managed Identity & Key Vault	21
9.1	The Problem: Credential Management	21
9.2	Solution: Managed Identity + Key Vault	21
10	Google Cloud Database Services	22
10.1	Cloud SQL	22
10.2	Cloud Spanner	22
10.3	Cloud Bigtable	23
10.4	BigQuery	23
10.5	The Definitive Comparison	24
10.6	Use-Case Decision Guide	24
10.7	Smart City Example — All Four Together	25
11	OpenStack Components	26
11.1	Nova: Compute Service	26
11.2	Neutron: Networking Service	26
11.3	Other Key OpenStack Services	27
12	Master Quick-Reference Sheet	29

1 Service Level Agreement (SLA)

1.1 What is an SLA?

A **Service Level Agreement (SLA)** is a formal, written contract between a *service provider* and a *customer* that defines the expected quality and scope of a service.

Simple Definition:

SLA = “We promise to deliver this service at this level of quality.”

It sets **clear expectations**, **measurable targets**, and **consequences** for failure.

An SLA answers three fundamental questions:

1. **What** service will be provided?
2. **How well** will it be provided (performance level)?
3. **What happens** if things go wrong (penalties/compensation)?

1.2 Real-Life Examples

1.2.1 Internet Service Provider (ISP)

When you buy a home broadband plan, your SLA might state:

Example ISP SLA:

- Download speed: **100 Mbps**
- Uptime guarantee: **99%** (max ~7 hours downtime/month)
- Support response time: **within 2 hours**
- Compensation: refund if uptime drops below 99%

1.2.2 Cloud Provider (AWS / Azure / GCP)

Example Cloud SLA (Amazon S3-style):

- Uptime: **99.9%** (about 8.7 hours downtime/year)
- Data backup: every **24 hours**
- Compensation: **service credits** if uptime drops

1.2.3 College IT System (e.g., Campus Portal)

Example Campus SLA:

- App availability: **24/7**
- Max server downtime: **1 hour/month**
- Bug resolution: within **48 hours**

- **Violation:** system crashes during exams = SLA breach

1.3 Key Components of an SLA

Component	Description
Service Description	What exactly is being provided (cloud storage, hosting, internet)
Performance Metrics	Uptime %, speed, response time, error rate
Responsibilities	Provider maintains infrastructure; customer pays and uses correctly
Penalties / Compensation	Refund, service credits, or free service on failure
Support Details	24/7 helpline, email support, max response time
Exclusions	Events not covered (e.g., natural disasters = <i>force majeure</i>)

What is NOT included in an SLA:

- ✗ Encryption algorithm details (e.g., AES, RSA)
- ✗ Internal key storage methods

*Reason: SLA covers **service performance**, not internal technical implementation.*

1.4 Understanding Uptime

Uptime = the percentage of time a service is fully operational.

$$\text{Downtime} = (1 - \text{Uptime}\%) \times \text{Total Time}$$

Uptime SLA	Per Day	Per Month (30d)	Per Year
99%	14.4 min	7.2 hours	87.6 hours
99.9%	1.44 min	43.2 min	8.76 hours
99.99%	8.6 sec	4.3 min	52.6 min
99.999%	0.86 sec	26 sec	5.26 min

1.5 SLA Calculation — Worked Example

Problem: A system receives **10,000 requests/hour** with a **99.5% success rate SLA**. How many failures are allowed per hour?

Step-by-step solution:

$$\begin{aligned}\text{Failure rate} &= 100\% - 99.5\% = 0.5\% \\ \text{Allowed failures/hour} &= 0.5\% \times 10,000 \\ &= \frac{0.5}{100} \times 10,000 \\ &= \boxed{50 \text{ failures/hour}}\end{aligned}$$

✓ **Answer:** The system is allowed to fail on up to **50 requests per hour** while still meeting the SLA.

1.6 Multi-Level SLA Architecture

In enterprise and cloud environments, SLAs are not one-size-fits-all. They operate at **three hierarchical levels**:

Level 1 — Corporate SLA

Applies to **all customers** uniformly.

Example: “We guarantee 95% uptime for all users on our platform.”

Level 2 — Customer SLA

Tailored for a **specific customer or organisation**.

Example: “Company X has a premium contract guaranteeing 98% uptime.”

Level 3 — Service SLA

MOST IMPORTANT

Tied to a **specific service or application**.

Example: “Database service uptime = 99.9%; API response time $\leq 200\text{ms}$.”

⇒ Service-level SLA is **legally binding** because it directly governs actual service delivery and is measurable.

Analogy (College context):

- Corporate level ⇒ Attendance policy for entire college: $\geq 75\%$
- Customer level ⇒ Your department rule: $\geq 80\%$
- Service level ⇒ This specific subject: $\geq 90\%$ (most specific, enforced)

Memory Trick:

SLA = Service + Level + Agreement = *Quality Promise Document*

1.7 Why SLA Matters

Without SLA ×	With SLA ✓
No clear rules	Clear, measurable expectations
No performance guarantee	Guaranteed performance levels
No compensation if things fail	Refund / credits on failure
No accountability	Both sides are accountable
Trust issues	Builds trust and confidence

1.8 Related Concepts: SLO and SLI

Acronym	Full Form	Meaning
SLA	Service Level Agreement	The <i>contract</i> — includes penalties
SLO	Service Level Objective	The <i>target</i> — e.g., 99.9% uptime
SLI	Service Level Indicator	The <i>measurement</i> — actual uptime recorded

How they relate: SLI measures reality, SLO sets the goal, SLA is the contract that includes the SLO.

1.9 Key SLA Metrics

Metric	Full Form	Example
RTO	Recovery Time Objective	Server must recover within 2 hours after failure
RPO	Recovery Point Objective	Data must not be lost beyond 1 hour back
MTTR	Mean Time To Repair	Average bug fix time = 30 minutes
MTBF	Mean Time Between Failures	System runs 500 hours before failing on average
Uptime %	Availability percentage	99.9% = 8.76 hours downtime/year

2 Parallel Databases vs Traditional Databases

2.1 Traditional (Single-Node) Database

A traditional database runs on a **single machine** (one server). All queries, storage, and computation happen on one node.

Limitations:

- ✗ Performance bottleneck (one CPU, one disk)
- ✗ Cannot handle petabyte-scale data
- ✗ Single point of failure
- ✗ Vertical scaling only (add more RAM/CPU to same machine)

When to use: Small to medium applications, college projects, small e-commerce websites.

2.2 Parallel Database

A parallel database distributes data and processing across **multiple nodes** (machines). Queries are broken into subtasks and executed simultaneously.

Key Features:

- ✓ Data is **partitioned** across nodes
- ✓ Queries run **in parallel** — multiple nodes work simultaneously
- ✓ Much faster for large datasets
- ✓ Horizontal scaling (add more nodes)
- ✓ Used in cloud environments (AWS Redshift, Google BigQuery)

2.3 Analogy

Traditional DB: 1 person solving 1000 math problems alone.

Parallel DB: 10 people each solving 100 problems simultaneously. ⇒ **10x faster!**

2.4 Comparison Table

Feature	Traditional DB	Parallel DB
Architecture	Single machine	Multiple nodes
Query Speed	Slower for large data	Faster (parallel execution)
Scalability	Vertical only	Horizontal (add nodes)
Fault Tolerance	Low (single point of failure)	High (data replicated)
Cost	Lower upfront	Higher (cloud infra needed)
Use Case	Small/medium apps	Big data, analytics, cloud
Example	MySQL on one server	Amazon Redshift, BigQuery

3 Real-Time vs Batch Processing

3.1 Batch Processing

Data is **collected over a period**, then processed all at once in a “batch.”

Characteristics:

- ✗ High latency (minutes to hours delay)
- ✓ Efficient for very large historical datasets
- ✓ Cheaper to run (off-peak hours)

Example: Bank statement generated at end of day, payroll processing

Tools: Apache Hadoop MapReduce, AWS S3 + Spark batch

3.2 Real-Time (Stream) Processing

Data is processed **instantly as it arrives**, with sub-second latency.

Characteristics:

- ✓ Sub-second latency
- ✓ Handles millions of events per second
- ✓ Essential for live monitoring, alerts, recommendations

Example: Swiggy live order tracking, stock market feeds, fraud detection

Tools: Apache Kafka + Flink, Azure Event Hubs, AWS Kinesis

3.3 Decision Rule

How to choose?

If **latency matters** (sub-second response needed) ⇒ Use **Real-Time Processing**

If **latency is acceptable** (minutes/hours OK) ⇒ Use **Batch Processing**

Exam Trap:

Requirement: “1 million events/sec with sub-second latency”

- ✗ AWS S3 + Spark batch = **NOT suitable** (high latency)
- ✓ Apache Kafka + Flink = **Correct choice**

3.4 Comparison Table

Feature	Batch Processing	Real-Time Processing
Latency	Minutes to hours	Milliseconds
Data Trigger	Scheduled interval	Each incoming event
Use Case	Reports, analytics	Monitoring, fraud, live feeds
Tools	Hadoop, Spark batch	Kafka, Flink, Kinesis
Cost	Lower	Higher
Complexity	Simpler	More complex

4 Object Storage for Large-Scale Data

4.1 The Problem with Traditional Storage

When dealing with petabytes of unstructured data (images, videos, logs, sensor data), traditional storage solutions fall short:

- **RDBMS** — designed for structured/tabular data only
- **Single disk / NAS** — not scalable, single point of failure
- **File systems** — directory-based, not designed for massive scale

4.2 Object Storage

Object storage stores data as **objects** (data + metadata + unique ID), not as files or blocks.

Why Object Storage for Big Data?

- ✓ Stores huge, unstructured files (videos, images, ML datasets)
- ✓ Designed for **massive scale** (petabytes to exabytes)
- ✓ Supports **parallel access** — multiple compute nodes can read simultaneously
- ✓ Geographically distributed and highly durable (11 nines durability on AWS S3)
- ✓ Cost-effective for large volumes

4.3 Examples

Provider	Service	Common Use Case
Amazon AWS	S3 (Simple Storage Service)	ML training data, backups, media
Google Cloud	GCS (Cloud Storage)	BigQuery staging, analytics
Microsoft Azure	Azure Blob Storage	Video streaming, archive

Real-World Example:

Netflix stores all movies and shows using object storage.

When you press Play, your device reads the video object directly from a distributed store — **fast, scalable, always available**.

5 MapReduce and Combiner Optimisation

5.1 MapReduce Overview

MapReduce is a programming model for processing **large datasets** in a distributed cluster.

Step 1. Map Phase: Input data is split and each chunk is processed by a mapper. Mapper emits key-value pairs.

Step 2. Shuffle & Sort: Framework groups all values by key across all mappers.

Step 3. Reduce Phase: Reducer aggregates grouped values into final output.

Example — Word Count:

Input: “apple orange apple banana apple”

Map output	After Shuffle	Reduce output
apple → 1	apple: [1,1,1]	apple: 3
orange → 1	orange: [1]	orange: 1
apple → 1	banana: [1]	banana: 1
banana → 1		
apple → 1		

5.2 The Problem: Disk Full During Reduce Phase

Scenario: Processing a **2 TB dataset**. Disk fills up during the reduce phase.

Root Cause: Too much intermediate data is being transferred from mappers to reducers without any pre-aggregation.

5.3 Solution: The Combiner

A **Combiner** is a **mini-reducer** that runs *locally* on each mapper’s output **before** the data is sent over the network to the reducer.

How Combiner Helps:

- ✓ Reduces the volume of intermediate data
- ✓ Less network traffic
- ✓ Less disk usage at the reducer stage

✓ Significantly faster overall job execution

Visualising the difference:

Without Combiner	With Combiner
apple → 1 apple → 1 apple → 1 (3 records sent)	apple → 3 (1 record sent)

Key Insight:

The Combiner is **optional** but critical for performance when dealing with large datasets. It's transparent to the reducer — the reducer sees the same aggregated output whether or not a combiner was used.

6 OpenStack CPU Overcommit

6.1 What is CPU Overcommit?

In cloud computing, **CPU overcommit** means allocating **more virtual CPUs (vCPUs) than physical CPU cores** available on the host machine.

Why is this possible?

Virtual machines rarely use 100% of their allocated CPU all the time. By overcommitting, the cloud provider can host more VMs per physical server, **increasing utilisation and revenue**.

6.2 The Formula

$$\text{Total vCPUs} = \text{Physical CPUs} \times \text{Overcommit Ratio}$$

6.3 Worked Example

Problem: OpenStack cluster has **900 physical CPUs** with an overcommit ratio of **4.5**. How many vCPUs can be allocated?

$$\begin{aligned}\text{Total vCPUs} &= 900 \times 4.5 \\ &= \boxed{4050 \text{ vCPUs}}\end{aligned}$$

✓ **Answer:** 4050 vCPUs can be offered to tenants.

6.4 Analogy

Overcommit is like a teacher teaching 4 classes:

One teacher (physical CPU) is assigned to 4 different classrooms (4 VMs). The teacher moves between rooms as needed — efficiently using time. Similarly, a physical CPU services multiple vCPUs by time-slicing.

Risk of High Overcommit:

If all VMs spike their CPU usage simultaneously, performance degrades for everyone — this is called *CPU contention*.

7 Azure SQL Database Compute Tiers

7.1 Overview

Azure SQL Database offers three distinct compute tiers, each designed for different workload characteristics:

7.2 General Purpose Tier

General Purpose — The balanced, cost-effective tier

- Uses **remote storage** (Azure Blob — separated from compute)
- Good read/write performance for most workloads
- Suitable for standard business applications
- *Example: A college portal storing student records and timetables*

7.3 Business Critical Tier

Business Critical — High performance, low latency

- Uses **local SSD storage** (physically attached to compute node)
- Very low I/O latency — data is right next to the processor
- **Does NOT use remote storage** (common exam misconception!)
- Multiple replicas for high availability
- *Example: A banking application where a 10ms delay causes financial risk*

7.4 Hyperscale Tier

Hyperscale — Unlimited, auto-scaling storage [EXAM IMPORTANT]

- Storage **automatically scales** — no predefined limit (beyond 100 TB)
- Rapid database snapshots with no performance impact
- Fast restore from any point in time
- Designed for massive, continuously growing datasets
- *Example: IoT platform collecting sensor data 24/7; Instagram storing billions of images*

7.5 Comparison Table

Feature	General Purpose	Business Critical	Hyperscale
Storage Type	Remote (Blob)	Local SSD	Distributed layers
Latency	Medium	Very Low	Low
Max Storage	4 TB	4 TB	Unlimited (auto)
Availability	Standard	High (replicas)	High
Cost	Low-Medium	High	High
Best For	General apps	Low-latency apps	Massive/growing data

8 Azure Virtual Machines — Disk Limits & Secure Access

8.1 VM Disk Limit — What Controls It?

When attaching data disks to an Azure VM, the **maximum number of disks** you can attach is determined by the **VM size (SKU)**, not by the operating system or region.

Think of it like a computer cabinet:

A small mid-tower has 2 drive bays; a full tower has 8 drive bays.

The **cabinet size (VM SKU)** determines how many disks you can fit.

Small VM (e.g., Standard_B2s) → max 4 data disks

Large VM (e.g., Standard_D64s) → max 64 data disks

Common Misconceptions — These do NOT control disk limits:

- ✗ Operating System (Windows/Linux)
- ✗ Azure Region (East US, West Europe, etc.)
- ✗ Subscription type

8.2 Secure Login to Linux VMs

8.2.1 Traditional Approach (Less Secure)

- SSH keys stored locally
- Public IP exposed on the VM
- Keys can be lost, stolen, or misconfigured

8.2.2 Best Practice: Azure AD SSH + Azure Bastion

Azure AD SSH Login:

- ✓ Uses your Microsoft/Azure identity — no local keys needed
- ✓ Login is tied to your identity, not a file
- ✓ Centrally managed; easy to revoke access

Azure Bastion:

- ✓ Acts as a secure, managed jump-server (gateway)
- ✓ The VM has **no public IP** — not directly reachable from the internet
- ✓ Connection happens through the browser/portal over HTTPS
- ✓ Eliminates surface area for SSH brute-force attacks

Analogy:

Without Bastion: You give everyone a door key (SSH key) and your home address (public IP).

With Bastion: Everyone enters through a single, guarded security checkpoint (Bastion) using biometrics (Azure AD). Your home has no visible door.

8.3 Azure App Service Deployment Slots

8.3.1 What are Deployment Slots?

Slots let you run multiple versions of your web app in the same App Service Plan:

- **Production slot** = live website users see
- **Staging slot** = new version being tested
- You **swap** them when the new version is ready

8.3.2 Slot Limits by Plan

App Service Plan	Max Slots	Key Feature
Free / Basic	0	No slots available
Standard	10	Basic slot swapping
Premium	20	Slot swapping + Traffic Warm-Up

8.3.3 Traffic Warm-Up (Premium Feature)

What is Warm-Up?

Before swapping staging → production, Azure **pre-loads** the new app version:

- Sends initialisation requests to staging
- Warms up application caches and connection pools
- Ensures the new version handles traffic without cold-start issues

Result: Zero-downtime, smooth deployment with no performance dip at swap time.

9 Azure Managed Identity & Key Vault

9.1 The Problem: Credential Management

Applications often need secrets — API keys, database passwords, connection strings. Storing these insecurely leads to breaches.

Bad Practices (Never do these):

- ✗ Hardcoding passwords in code: `password = "MyP@ss123"`
- ✗ Storing in environment variables visible to all team members
- ✗ Committing secrets to Git repositories

9.2 Solution: Managed Identity + Key Vault

How it works:

Step 1: App is assigned a **Managed Identity** by Azure (automatic, no credentials needed by the app itself)

Step 2: Secrets (passwords, API keys) are stored securely in **Azure Key Vault**

Step 3: App authenticates to Key Vault using its Managed Identity — **no passwords stored anywhere in code**

- ✓ No secrets in code, config files, or environment variables
- ✓ Access is centrally controlled and audited
- ✓ Works seamlessly with Azure services (VMs, Functions, App Service)

Analogy:

Instead of writing your locker combination on a sticky note and sticking it on your monitor (hardcoded password), you use biometric access to your locker (Managed Identity). The locker contains your valuables (secrets in Key Vault).

10 Google Cloud Database Services

Overview: Google Cloud offers four major database services, each optimised for a completely different use-case. Understanding *when* to use each is more important than memorising features.

10.1 Cloud SQL

Cloud SQL — Managed Relational Database (Standard Apps)

What it is: A fully managed version of popular relational databases hosted on Google Cloud.

Supported engines: MySQL • PostgreSQL • SQL Server

Features:

- Supports SQL queries, joins, transactions, foreign keys
- Fully managed: Google handles backups, patching, replication
- Vertical scaling (increase machine size)
- Regional deployment (single region)

Limitations:

- Does not scale globally (not multi-region by default)
- Limited to a few terabytes
- Not suitable for billions of rows at high throughput

Example: Student management system, e-commerce backend, blog platform

10.2 Cloud Spanner

Cloud Spanner — Globally Distributed Relational Database

What it is: Think of it as MySQL/PostgreSQL + global scale + strong consistency. It is the world's first globally distributed, strongly consistent relational database.

Features:

- Full SQL support (tables, joins, indexes)
- **ACID transactions** — bank-level reliability across regions
- **Strong consistency** — all regions always see the same data
- Automatically replicates across multiple regions worldwide
- Horizontal scaling

When to use: When you need SQL + global scale + strict consistency

Examples:

- Google Pay — global payment transactions
- Airline booking system — inventory must be consistent globally
- Multiplayer gaming — global leaderboard with consistency

Why not Cloud SQL? Cloud SQL is regional. If a user in India and a user in USA both book the last airline seat at the same moment, Cloud SQL might allow a double-booking. Cloud Spanner prevents this.

10.3 Cloud Bigtable

Cloud Bigtable — NoSQL Wide-Column Store (Massive Scale, Fast I/O)

What it is: A highly scalable NoSQL database for workloads requiring very fast reads/writes at massive scale. Originally built by Google for internal use (Gmail, Google Search index).

Features:

- Row-key based access (no joins, no foreign keys)
- Handles petabytes of data across thousands of nodes
- Millisecond read/write latency
- Optimised for time-series data (data ordered by timestamp)
- Integrates with Hadoop/Spark for analytics

Limitations:

- No SQL queries
- No complex joins or relationships
- Not suitable for ACID transactions

Examples:

- IoT sensors sending readings every second
- Google Maps storing real-time location data
- Stock market price tick data
- Smart city monitoring system

10.4 BigQuery

BigQuery — Serverless Data Warehouse for Analytics [**NOT for app storage**]

What it is: A fully managed, serverless data warehouse designed for running **analytical SQL queries** on enormous datasets (terabytes to petabytes).

Features:

- Serverless — no infrastructure to manage
- SQL queries on terabyte/petabyte datasets in seconds
- **Columnar storage** — reads only relevant columns (very fast for analytics)
- Integrates with Data Studio, Looker, ML (BigQuery ML)

- Pay-per-query or flat-rate pricing

When NOT to use it:

- Real-time transactional workloads
- Application backend database
- Low-latency single-row lookups

Examples:

- “Find total sales for the last 5 years across all countries”
- “Analyse user behaviour patterns from 2 billion events”
- Business intelligence dashboards

10.5 The Definitive Comparison

Feature	Cloud SQL	Cloud Spanner	Bigtable	BigQuery
Type	Relational	Relational	NoSQL (wide-col)	Data Warehouse
SQL Support	✓	✓	✗	✓
Joins	✓	✓	✗	✓
ACID Transactions	✓	✓ (global)	✗	✗
Global Scale	✗	✓	✓	✓
Latency	Low	Low	ms (fastest)	Seconds (queries)
Max Scale	TBs	PBs	PBs	PBs
Use Case	Standard apps	Global OLTP	High-speed I/O	Analytics

10.6 Use-Case Decision Guide

Choose the right tool — quick decision guide:

- “Normal app backend (like college portal)” ⇒ **Cloud SQL**
- “Global app, transactions must be consistent” ⇒ **Cloud Spanner**
- “Millions of writes/sec, IoT, time-series” ⇒ **Cloud Bigtable**
- “Analyse huge historical dataset, BI reports” ⇒ **BigQuery**

10.7 Smart City Example — All Four Together

Smart City Unified Platform:

Sensor readings every second (IoT) ⇒ **Cloud Bigtable**

City app backend (residents, services) ⇒ **Cloud SQL**

Global payment for city services ⇒ **Cloud Spanner**

City analytics dashboard (trends) ⇒ **BigQuery**

Same project, four different databases — each the best for its job.

11 OpenStack Components

11.1 Nova: Compute Service

Component	Role
Nova Scheduler	Decides which host a new VM should run on (placement logic)
Nova Compute	Actually runs the VM on the selected host using hypervisor
Nova API	Accepts user/operator requests and routes them internally
Nova Conductor	Coordinates complex operations across Nova components

Analogy for VM Placement:

Nova Scheduler = Hostel warden who decides which room you get

Nova Compute = The actual room (physical server running your VM)

11.2 Neutron: Networking Service

Component	Role
DHCP Agent	Automatically assigns IP addresses to VMs
L3 Agent	Handles routing between subnets and external networks
ML2 Plugin	Core plugin managing network types (VLAN, VXLAN, etc.)

Issue: VMs communicate but have no IP addresses

Root Cause: DHCP Agent failure — the DHCP agent is responsible for IP allocation. If it crashes, VMs can connect to the network (L2 working) but receive no IP address.

Analogy: Roads are open (network works), but the address allotment office (DHCP) is closed — no one gets assigned an address.

11.3 Other Key OpenStack Services

Project	Service	Function
Keystone	Identity	Authentication and authorisation for all services
Glance	Image	Stores and retrieves VM images (OS templates)
Cinder	Block Storage	Persistent block volumes attached to VMs
Swift	Object Storage	Distributed object store (like AWS S3)
Horizon	Dashboard	Web UI for managing all OpenStack services
Heat	Orchestration	Template-based stack deployment (like CloudFormation)

12 Master Quick-Reference Sheet

Topic	Core Concept	Exam Key Point
SLA	Quality promise between provider and customer	Includes RTO, MTTR, penalties — NOT encryption
SLO	Target metric inside an SLA	Service-level SLA is legally binding
Uptime 99%	≈ 7.2 hours downtime/month	Higher uptime % = better reliability
RTO	Max recovery time after failure	Must be specified in SLA
MTTR	Average time to repair an issue	Shorter MTTR = better service
Multi-level SLA	Corporate $\supset$ Customer $\supset$ Service	Service-level is most specific and enforceable
Parallel DB	Multi-node, distributed queries	Faster than single-node for large data
Real-Time	Sub-second, event-driven	Use Kafka + Flink, not Spark batch
Batch	Periodic bulk processing	Use for reports, not live data
Object Storage	Unstructured data at scale	AWS S3, GCS — best for big unstructured data
Combiner	Pre-aggregates mapper output	Reduces disk I/O during reduce phase
Overcommit	vCPUs > physical CPUs	Formula: Physical $\times$ Ratio
Azure SQL GP	Remote storage, cost-effective	Not for ultra-low latency
Azure SQL BC	Local SSD, low latency	Does NOT use remote storage
Azure Hyperscale	Auto-scaling, unlimited storage	Best for rapidly growing data
VM Disk Limit	Controlled by VM SKU	NOT OS, NOT region, NOT subscription
Bastion	Secure gateway, no public IP	Prevents direct SSH brute-force
Key Vault	Secure secret storage	Use with Managed Identity, not hardcoded
Cloud SQL	Managed MySQL/PostgreSQL	Simple apps, single region
Cloud Spanner	Global SQL + ACID	Payments, bookings, global transactions
Cloud Bigtable	NoSQL, ms latency, time-series	IoT, logs, sensor data

Five Golden Rules for the Exam:

1. SLA is about **service performance**, not internal technical details (no encryption algorithms).
2. Service-level SLA is the most specific and is **legally binding**.
3. Choose database by use-case: SQL + global = Spanner; IoT = Bigtable; analytics = BigQuery; simple = Cloud SQL.
4. For real-time (sub-second latency), batch processing is **always wrong**.
5. Combiner is the fix for disk-full issues in large MapReduce jobs.